



High Performance Computing Exercise Sheet 10

HS 25
Dr. Douglas Potter

<http://www.astro.uzh.ch/>

Teaching Assistants:

Cesare Cozza, cesare.cozza@uzh.ch

Mark Eberlein, mark.eberlein@uzh.ch

Issued: 21.11.2025

Due: 28.11.2025

Exercise 1 [CUDA: Compile and run on Daint]

Follow this instructions to compile and run `hello_world.cu`, that you can find in the **Example** folder.

- (i) Load the proper modules:

```
module load cray
module load cudatoolkit
module swap PrgEnv-cray PrgEnv-gnu
```

- (ii) Compile the example code:

```
nvcc hello_world.cu
```

- (iii) In order to run on a GPU, we first need to allocate a node with a GPU. We will launch the bash terminal on a node:

```
srun -A uzh8 --time=00:30:00 --partition=debug --pty bash
```

- (iv) Once the job has been allocated we can proceed to execute the code on the node:

```
./a.out
```

Exercise 2 [Basic operations with CUDA]

The material for this exercise is in the `sandbox1` folder.

The code `setvec.cu` is supposed to invoke the CUDA kernel `kern_set_val` to set the values of a pointer on the GPU, then copy it on the host memory. You have to write the missing lines of code, indicated by the comments `//TO DO: . . .`.

If your implementation is correct, the program should output `'PASSED !!'`.

Commit: Push your implementation to your repository. Make sure that your implementation works also when `N` is not divisible by `BLOCKSIZE`.

Exercise 3 [Summing matrices]

The material for this exercise is in the `sandbox2` folder.

Here, the code `addongpu.cu` first initializes two matrices `mat_in1` and `mat_in2` of size $n_x \times n_y$ with random numbers on the host. Then it should copy these matrices on the GPU and use the CUDA kernel `kernadd` to compute the sum `mat_in1+mat_in2`. Again, some lines of code have been deleted, you have to write them back.

If the output is 'PASSED', your implementation is correct.

Bonus: - Rewrite this for 2D blocks (using both x and y values for `threadIdx`, `BlockIdx` and `BlockDim`).

Hint: You now have to define a number of threads in each direction, try with 16×16 . Search for `dim3` in <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html> in order to properly modify the kernel launch.

Commit: Push your implementation to your repository.

Exercise 4 [Convolutions]

In the last exercise you will have to complete the code located in the `convol` folder.

In this case, the `main.c` file calls external CUDA routines written in the `convol.cu` file. In order to compile such a code, use the `Makefile` provided.

The code computes the convolution between a $N \times N$ random matrix and a 3×3 unweighted kernel (without corners). This operation is done both on the GPU and on the CPU. Once again, complete the program where requested (`//TODO:...`) and compare the performance (timing) between the GPU-parallel computation and the CPU-serial one.

Bonus: Find the optimal size of blocks for a given size `N` of the matrix.

Commit: Push your implementation to your repository.